EXERCISE 01

Flavio Massimiliano Ricci
ricci.2083804@studenti.uniroma1.it

October 6, 2026

Contents

1 Introduction **1**

2 Methodology **1**
2.1 Fit . . . 1
2.2 Performance measures . . . 1

3 Results **2**
3.1 Case I: $P = 10$, $\sigma = 0$, $P_{\text{test}} = 20$ . . . 2
3.1.1 Function A . . . 2
3.1.2 Function B . . . 2
3.2 Case II: $P = 100$, $\sigma = 1$, $P_{\text{test}} = 20$ . . . 3
3.2.1 Function A . . . 3
3.2.2 Function B . . . 4
3.3 Case III: $P = 10000$, $\sigma = 1$, $P_{\text{test}} = 100$ . . . 4
3.3.1 Function A . . . 4
3.3.2 Function B . . . 5

4 Conclusions **5**

1 Introduction

Solution of exercise 01 on polynomial regression.

2 Methodology

2.1 Fit

The data were fitted using numpy.polyfit from python's library numpy, which uses the least squares method. It should be noted that in case I we have 10 pairs of data, but a polynomial of degree 10 has 11 parameters; so the problem is undetermined, because there are infinitely many degree-10 polynomials that can fit the data. However numpy.polyfit selects one solution among the family of possible solutions. [1]

2.2 Performance measures

Performance was evaluated using the mean squared error.
The test mean squared error is computed using the noisy test observations:

$$\text{MSE}_{\text{test}} = \frac{1}{P_{\text{test}}} \sum_{\mu=1}^{P_{\text{test}}} \left[y_\mu^{\text{test}} - \hat{f}_d(x_\mu^{\text{test}}) \right]^2 . \tag{1}$$

[1] In particular it computes the Vandermonde matrix, rescales it, and uses numpy.linalg.lstsq to solve the equation (a @ x = b). It chooses the x that minimizes the Euclidean 2-norm $||b - ax||$; if there are multiple minimizing solutions, the one with the smallest 2-norm $||x||$ is returned. This is why a fit was obtained even in that case.

Even if $\hat{f}_d(x) = f(x)$, since $y_\mu = f(x_\mu) + \eta_\mu$, we would still have $\mathrm{MSE}_{\mathrm{test}} \approx \sigma^2$.
However the generating function is known, so we can also evaluate the performance comparing the fit directly with the hidden function.

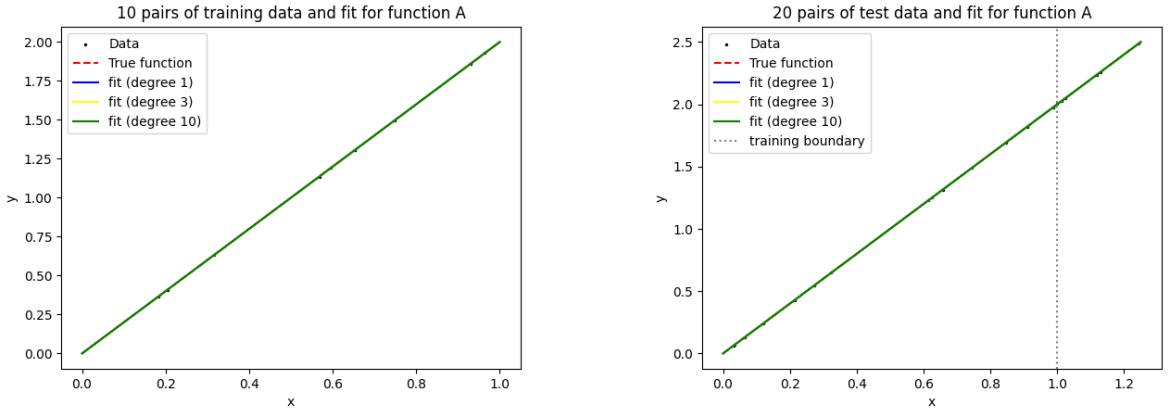
$$\mathrm{MSE}_f = \frac{1}{P_{\mathrm{test}}} \sum_{\mu=1}^{P_{\mathrm{test}}} \left[f(x_\mu^{\mathrm{test}}) - \hat{f}_d(x_\mu^{\mathrm{test}}) \right]^2 . \tag{2}$$

Both quantities are also evaluated separately for $x \leq 1$ ("in") and $x > 1$ ("out"), because the training set is generated in the interval $[0, 1]$ but the test set is generated in the interval $[0, 1.25]$. In this way we can differentiate between interpolation and extrapolation.

3 Results

3.1 Case I: $P = 10$, $\sigma = 0$, $P_{\mathrm{test}} = 20$

3.1.1 Function A



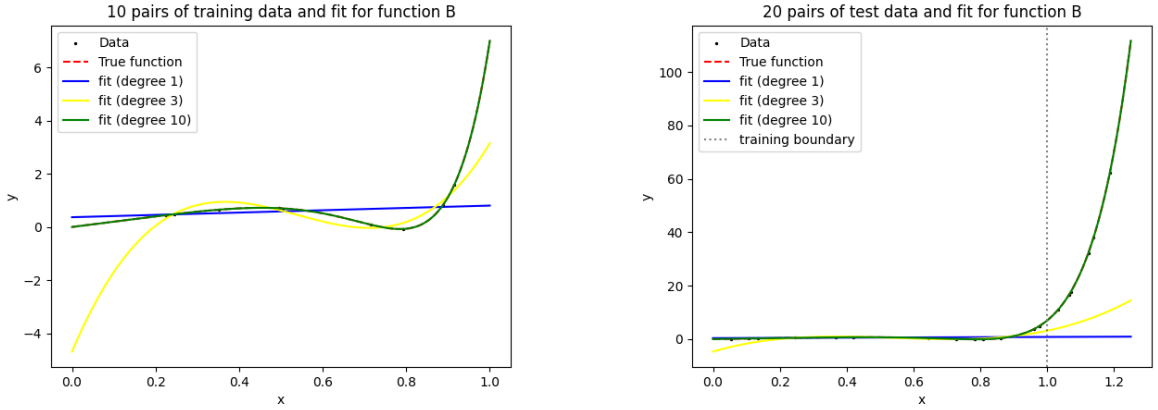
(a) Training data and fitted models.

(b) Test data, fitted models, and training boundary.

Figure 1: Function A for Case I.

In the noiseless case the performance of all three models is excellent, especially the degree-1 model which has almost machine-precision errors.

3.1.2 Function B



(a) Training data and fitted models.

(b) Test data, fitted models, and training boundary.

Figure 2: Function B for Case I.

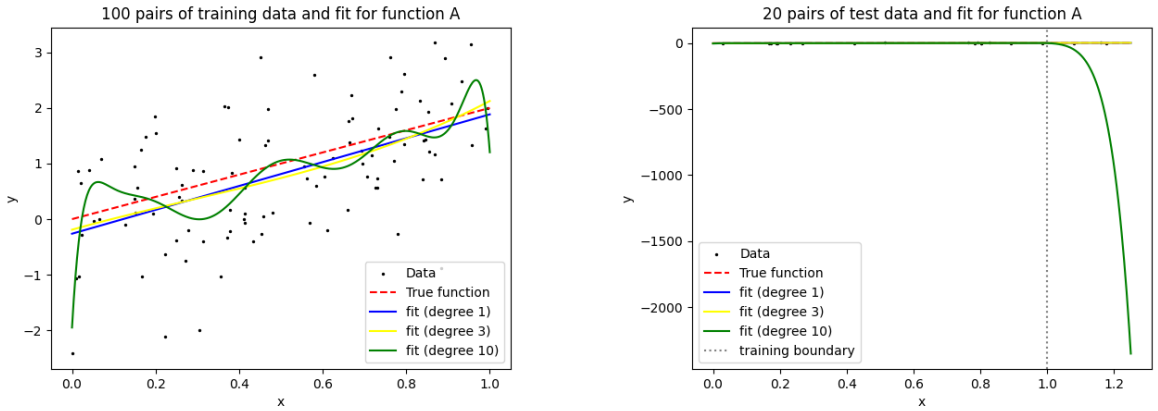
The degree-1 and degree-3 models show strong underfitting, therefore their large errors are due to model bias.
The degree-10 model reconstructs the function almost perfectly. However, keeping in mind what was said in section 2.1, this result should be interpreted with care; this fit is actually undetermined.

Table 1: Mean squared errors for Case I: $P = 10$, $\sigma = 0$, $P_{\text{test}} = 20$. "In" denotes $x \leq 1$ and "out" denotes $x > 1$.

Function	Degree	MSE_{test}	$\text{MSE}_{\text{test,in}}$	$\text{MSE}_{\text{test,out}}$	MSE_f	$\text{MSE}_{f,\text{in}}$	$\text{MSE}_{f,\text{out}}$
A	1	3.236×10^{-33}	4.622×10^{-33}	0	3.236×10^{-33}	4.622×10^{-33}	0
A	3	2.332×10^{-30}	1.472×10^{-30}	4.339×10^{-30}	2.332×10^{-30}	1.472×10^{-30}	4.339×10^{-30}
A	10	5.168×10^{-14}	9.435×10^{-16}	1.701×10^{-13}	5.168×10^{-14}	9.435×10^{-16}	1.701×10^{-13}
B	1	3.401×10^{2}	2.035	1.129×10^{3}	3.401×10^{2}	2.035	1.129×10^{3}
B	3	2.266×10^{2}	1.611	7.516×10^{2}	2.266×10^{2}	1.611	7.516×10^{2}
B	10	1.957×10^{-8}	1.573×10^{-10}	6.487×10^{-8}	1.957×10^{-8}	1.573×10^{-10}	6.487×10^{-8}

3.2 Case II: $P = 100$, $\sigma = 1$, $P_{\text{test}} = 20$

3.2.1 Function A



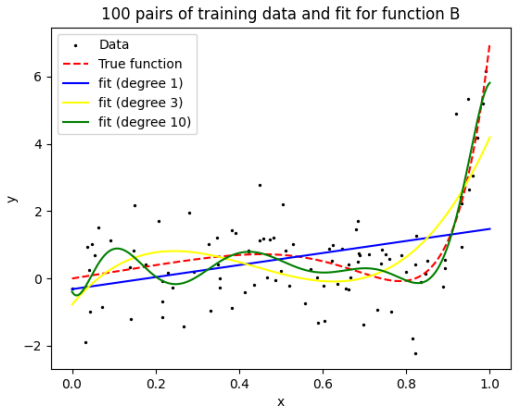
(a) Training data and fitted models.

(b) Test data, fitted models, and training boundary.

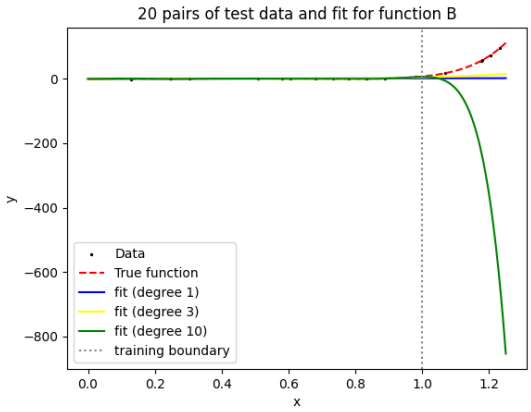
Figure 3: Function A for Case II.

All three models describe the function well, inside the training domain, according to $\text{MSE}_{f,\text{in}}$.
In particular the degree-1 model gives great results. The degree-3 model starts overfitting.
The degree-10 model overfits: its very large mean squared error is due to the contribution outside the training interval. The model fits part of random fluctuations of the data (having more parameters than necessary allows it to do so); in fact even visually we can see that despite having a low $\text{MSE}_{f,\text{in}}$ the function still doesn't look like a straight line. This is how the noise affects the results.
The primary cause for the enormous MSE is the extrapolation instability: high-order polynomial fits are strongly unstable, so even though they do a good job at interpolation, it doesn't mean they perform well in extrapolation.

3.2.2 Function B



(a) Training data and fitted models.



(b) Test data, fitted models, and training boundary.

Figure 4: Function B for Case II.

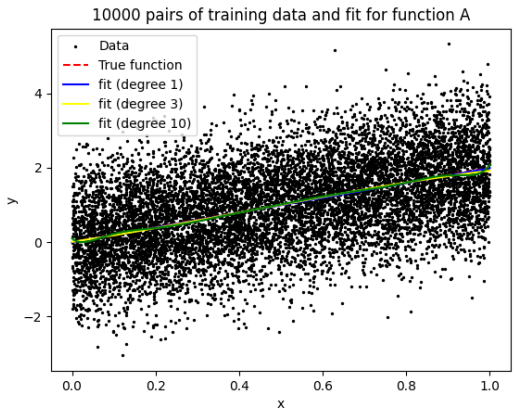
The underfitting of the low degree models is still visible. The degree-10 model is the closest to the real function, but the fit is extremely unstable; in fact outside the training domain it diverges in the opposite direction with respect to the real function. So 100 samples are still too few, for the given noise, to correctly extrapolate the function.

Table 2: Mean squared errors for Case II: $P = 100$, $\sigma = 1$, $P_{\text{test}} = 20$.

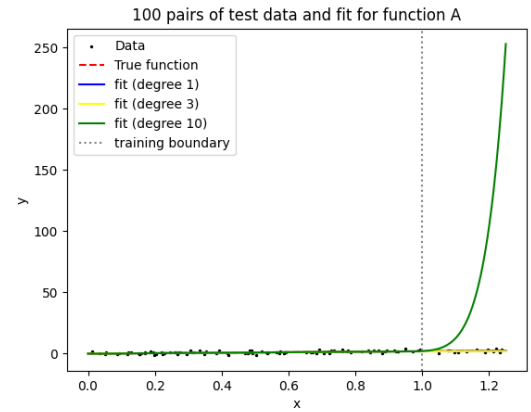
Function	Degree	MSE_{test}	$\text{MSE}_{\text{test,in}}$	$\text{MSE}_{\text{test,out}}$	MSE_{f}	$\text{MSE}_{f,\text{in}}$	$\text{MSE}_{f,\text{out}}$
A	1	1.270	1.472	0.1295	0.03242	0.03657	0.008900
A	3	1.326	1.530	0.1654	0.06359	0.03436	0.2292
A	10	2.496×10^4	1.502	1.664×10^5	2.495×10^4	0.02988	1.663×10^5
B	1	1.020×10^3	3.626	3.393×10^3	1.013×10^3	3.407	3.367×10^3
B	3	7.411×10^2	1.279	2.467×10^3	7.340×10^2	0.9258	2.444×10^3
B	10	4.599×10^4	1.203	1.533×10^5	4.592×10^4	0.2109	1.531×10^5

3.3 Case III: $P = 10000$, $\sigma = 1$, $P_{\text{test}} = 100$

3.3.1 Function A



(a) Training data and fitted models.

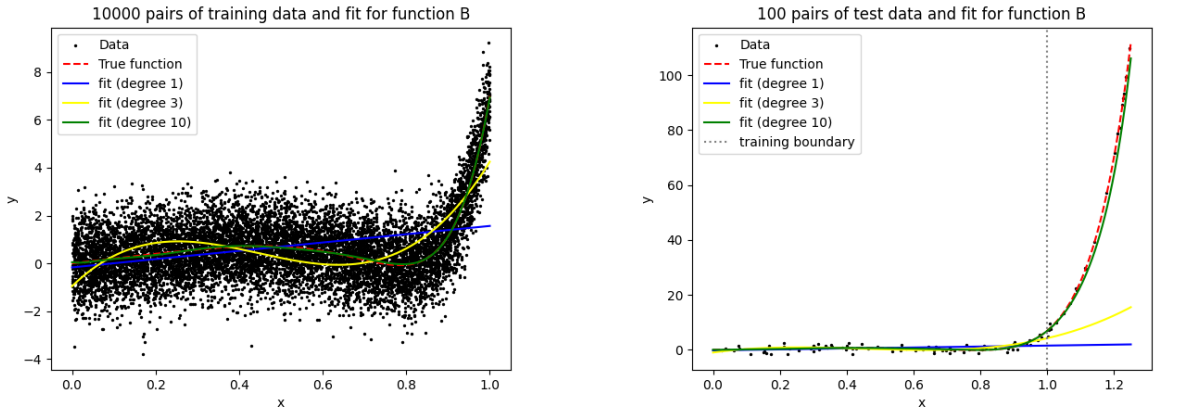


(b) Test data, fitted models, and training boundary.

Figure 5: Function A for Case III.

Increasing the number of data improves the performance of the correct degree-1 model.
The degree-10 model is accurate in interpolation and unstable in extrapolation. Due to the enormous amount of data the model replicates the function well inside the training domain; but the small coefficient errors induced by the noise are strongly amplified outside the training domain.

3.3.2 Function B



(a) Training data and fitted models.

(b) Test data, fitted models, and training boundary.

Figure 6: Function B for Case III.

Increasing the dataset allows the degree-10 model to correctly fit the function and have an acceptable mean squared error even outside the training interval.
The lower-degree models still underfit and retain an approximation bias.

Table 3: Mean squared errors for Case III: $P = 10000$, $\sigma = 1$, $P_{\text{test}} = 100$.

Function	Degree	MSE_{test}	$\text{MSE}_{\text{test,in}}$	$\text{MSE}_{\text{test,out}}$	MSE_{f}	$\text{MSE}_{f,\text{in}}$	$\text{MSE}_{f,\text{out}}$
A	1	1.039	1.000	1.301	9.286×10^{-5}	7.929×10^{-5}	1.837×10^{-4}
A	3	1.052	1.005	1.364	0.007661	4.748×10^{-4}	0.05575
A	10	1.300×10^{3}	1.012	9.995×10^{3}	1.303×10^{3}	9.600×10^{-4}	1.002×10^{4}
B	1	7.002×10^{2}	1.622	3.177×10^{3}	6.912×10^{2}	0.8191	3.139×10^{3}
B	3	5.120×10^{2}	1.169	2.323×10^{3}	5.040×10^{2}	0.2695	2.290×10^{3}
B	10	5.562	0.9679	21.85	4.004	0.001065	18.20

4 Conclusions

These examples show the problems of underfitting and overfitting, the bias-variance tradeoff and the difference between interpolation and extrapolation.